Bases de données relationnelles et SQL

Stocker, organiser et interroger des données — de SQLite à pandas

BD Modèle relationnel

PK Clés et contraintes

SQL Interroger et modifier

Py SQL depuis Python

Module POO – BD · 3A · 2026–2027

Pourquoi une base de données ?

Des fichiers CSV…

- aucune garantie d'unicité : deux lignes pour le même client
- aucune vérification des liens : une commande peut citer un client inexistant
- pas de types imposés : `"12,5"`, `12.5` et `douze` cohabitent
- accès concurrents : deux personnes modifient le même fichier
- tout relire pour retrouver une ligne



… à une base de données

- **clés primaires** : unicité garantie
- **clés étrangères** : intégrité des liens
- **types et contraintes** (`NOT NULL`, `CHECK`)
- **transactions** : modifications sûres et concurrentes
- **index** : accès rapide, un langage standard (**SQL**)

Le SGBD **refuse** les données incohérentes au lieu de laisser l'analyse les découvrir trop tard.

Plan du cours

1. **Modèle relationnel** — tables, lignes, colonnes, schéma — l'exemple client_commande
2. **Clés et contraintes** — clé primaire, clé étrangère, intégrité référentielle, NOT NULL, CHECK
3. **Le langage SQL** — SELECT, jointures, sous-requêtes, INSERT, UPDATE, DELETE, GROUP BY, NULL
4. **SQL depuis Python** — sqlite3, pandas.read_sql, correspondances SQL ↔ pandas, index
5. **SGBD et enjeux actuels** — fondements, familles de SGBD, défis

01

Le modèle relationnel

tables · lignes et colonnes · schéma et contenu · l'exemple client_commande

Notions de base

- **Base de données** = ensemble de tables
- **Table** = lignes + colonnes
- **Ligne** = suite de valeurs (un objet)
- **Colonne** = une propriété, avec un **type**
- **Schéma** = tables + colonnes + contraintes
- **Contenu** = l'ensemble des lignes à un instant t

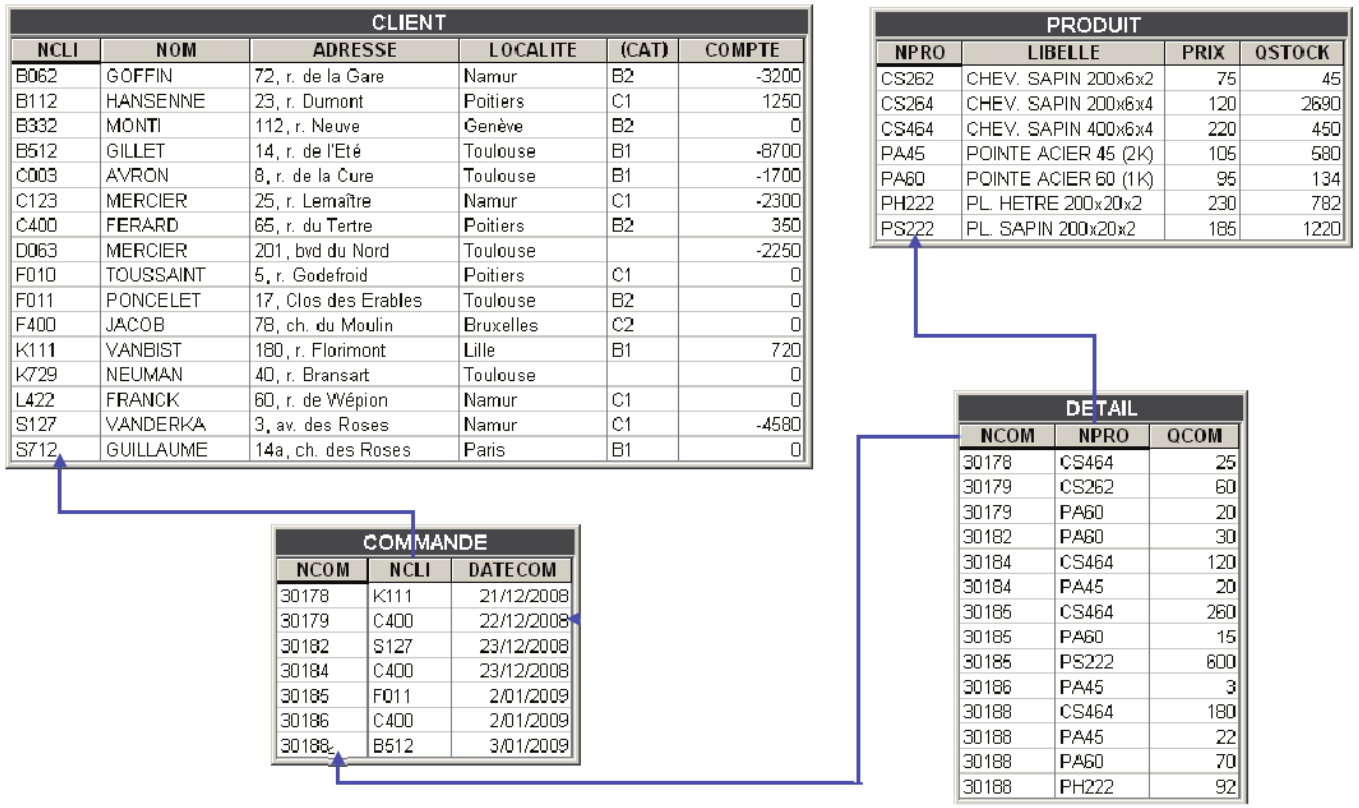
`table client`

↓ une colonne = une propriété, avec un type

NCLI	NOM	LOCALITE	COMPTE
B062	GOFFIN	Namur	-3200
B112	HANSENNE	Poitiers	1250
B332	MONTI	Genève	0
B512	GILLET	Toulouse	-8700

↑ une ligne = un client (n-uplet)

Exemple fil rouge : la base client_commande



CLIENT

NCLI	NOM	ADRESSE	LOCALITE	(CAT)	COMPTE
B062	GOFFIN	72, r. de la Gare	Namur	B2	-3200
B112	HANSENNE	23, r. Dumont	Poitiers	C1	1250
B332	MONTI	112, r. Neuve	Genève	B2	0
B512	GILLET	14, r. de l'Eté	Toulouse	B1	-8700
C003	AVRON	8, r. de la Cure	Toulouse	B1	-1700
C123	MERCIER	25, r. Lemaître	Namur	C1	-2300
C400	FERARD	65, r. du Tertre	Poitiers	B2	350
D063	MERCIER	201, bvd du Nord	Toulouse		-2250
F010	TOUSSAINT	5, r. Godefroid	Poitiers	C1	0
F011	PONCELET	17, Clos des Erables	Toulouse	B2	0
F400	JACOB	78, ch. du Moulin	Bruxelles	C2	0
K111	VANBIST	180, r. Florimont	Lille	B1	720
K729	NEUMAN	40, r. Bransart	Toulouse		0
L422	FRANCK	60, r. de Wépion	Namur	C1	0
S127	VANDERKA	3, av. des Roses	Namur	C1	-4580
S712	GUILLAUME	14a, ch. des Roses	Paris	B1	0

PRODUIT

NPRO	LIBELLE	PRIX	QSTOCK
CS262	CHEV. SAPIN 200x6x2	75	45
CS264	CHEV. SAPIN 200x6x4	120	2690
CS464	CHEV. SAPIN 400x6x4	220	450
PA45	POINTE ACIER 45 (2K)	105	580
PA60	POINTE ACIER 60 (1K)	95	134
PH222	PL. HETRE 200x20x2	230	782
PS222	PL. SAPIN 200x20x2	185	1220

COMMANDE

NCOM	NCLI	DATECOM
30178	K111	21/12/2008
30179	C400	22/12/2008
30182	S127	23/12/2008
30184	C400	23/12/2008
30185	F011	2/01/2009
30186	C400	2/01/2009
30188	B512	3/01/2009

DETAIL

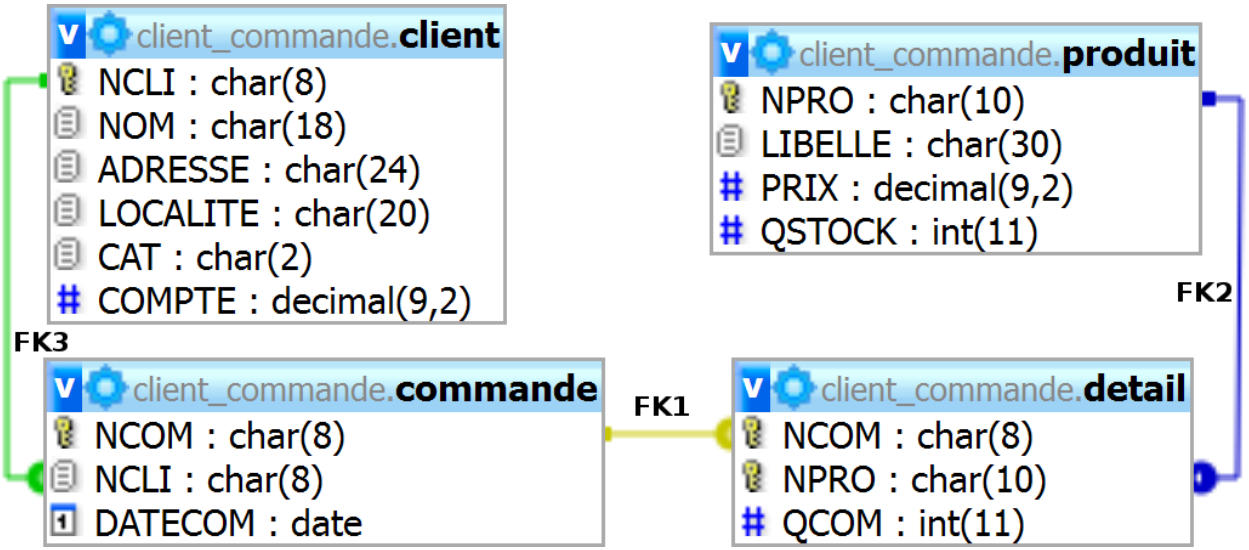
NCOM	NPRO	QCOM
30178	CS464	25
30179	CS262	60
30179	PA60	20
30182	PA60	30
30184	CS464	120
30184	PA45	20
30185	CS464	260
30185	PA60	15
30185	PS222	600
30186	PA45	3
30188	CS464	180
30188	PA45	22
30188	PA60	70
30188	PH222	92

4 tables

- `client` : 16
- `produit` : 7
- `commande` : 7
- `detail` : 14

Les flèches relient une colonne à la ligne qu'elle **référence**.

Schéma de la base client_commande

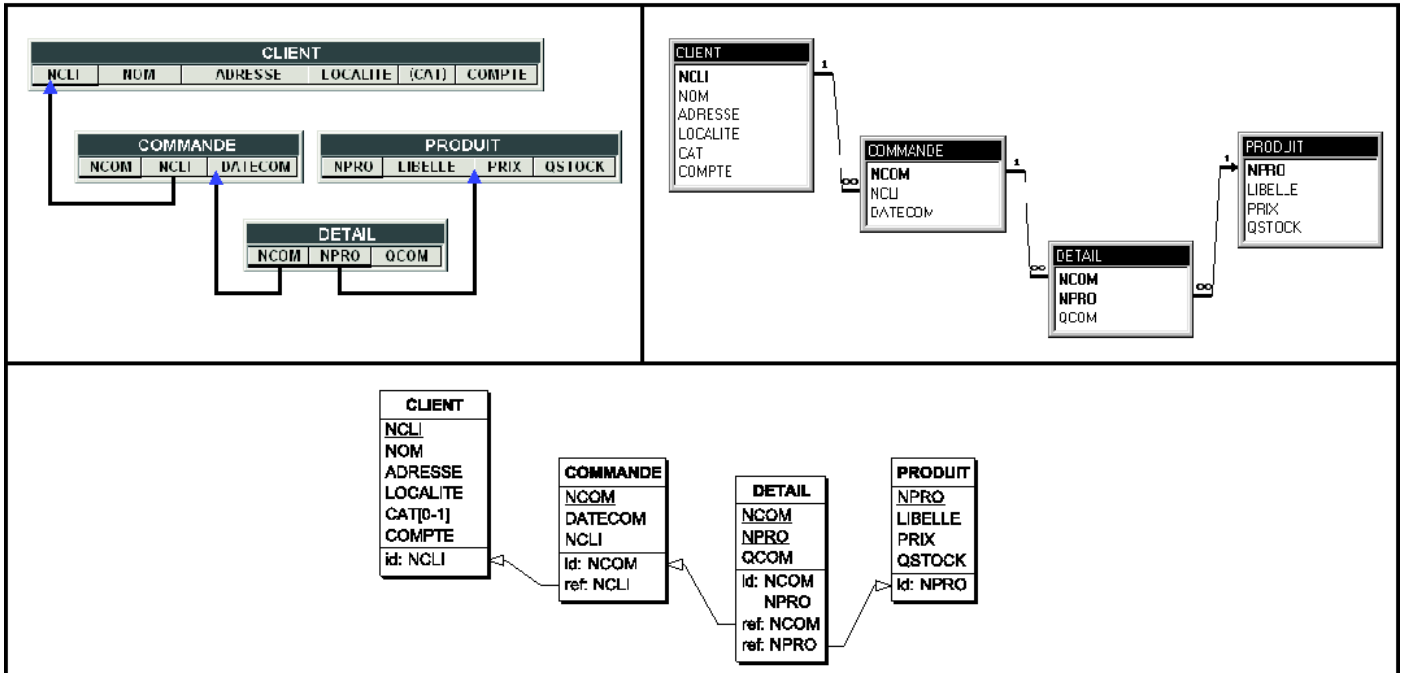


Le schéma décrit pour chaque table :

- ses colonnes et leur **type**
- sa **clé primaire** (🔑)
- ses **clés étrangères** (FK1, FK2, FK3)

`detail` relie `commande` et `produit` : une commande contient **plusieurs** produits, un produit apparaît dans **plusieurs** commandes.

Un même schéma, plusieurs représentations



Ce qui compte : les **tables**, leurs **clés** et les **liens** entre elles — pas la notation.

02

Clés et contraintes

clé primaire · clé étrangère · intégrité référentielle · NOT NULL, UNIQUE, CHECK

La clé primaire

Une ou **plusieurs** colonnes qui identifient **une ligne de manière unique**.

- impose une **contrainte d'unicité** : le SGBD rejette toute ligne qui reprend une valeur existante
- ne peut pas être **`NULL`**
- recommandée sur **toute** table
- sert de cible aux **clés étrangères**

Quelle est la clé primaire ?

Table	Clé primaire
client	`NCLI`
produit	`NPRO`
commande	`NCOM`
detail	`(NCOM, NPRO)`

Clé composite

Dans **`detail`**, ni **`NCOM`** ni **`NPRO`** seuls ne sont uniques : c'est le **couple** qui l'est.

Clé primaire : la contrainte d'unicité

SQL

```sql
INSERT INTO produit (NPRO, LIBELLE, PRIX, QSTOCK)
VALUES ('PH222', 'PL. HETRE 100x20x2', 120, 500);
```

Résultat

✖ `Erreur : UNIQUE constraint failed: produit.NPRO`

Le produit **PH222** existe déjà : la ligne est **rejetée** et la table n'est pas modifiée.

Table produit (extrait)

NPRO	LIBELLE
PA45	POINTE ACIER 45 (2K)
PA60	POINTE ACIER 60 (1K)
PH222	PL. HETRE 200x20x2
PS222	PL. SAPIN 200x20x2

Clé composite : unicité du couple (NCOM, NPRO)

Couple nouveau

SQL

```sql
INSERT INTO detail (NCOM, NPRO, QCOM)
VALUES ('30186', 'PH222', 5);
```

Résultat

✓ `OK — 1 ligne(s) affectée(s)`

Lignes de la commande 30186 avant insertion

NCOM	NPRO	QCOM
30186	PA45	3

Couple déjà présent

SQL

```sql
INSERT INTO detail (NCOM, NPRO, QCOM)
VALUES ('30186', 'PA45', 5);
```

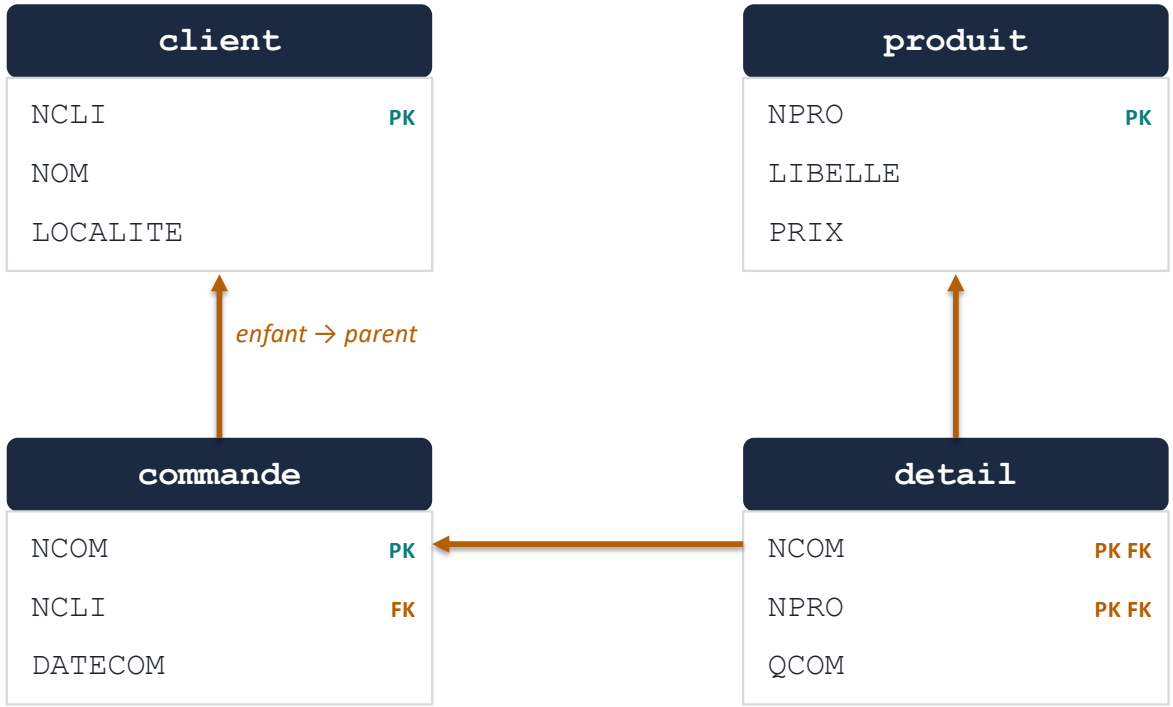
Résultat

✖ `Erreur : UNIQUE constraint failed: detail.NCOM, detail.NPRO`

La clé étrangère

Une colonne d'une table **enfant** qui fait référence à la **clé primaire** d'une table **parent**.

- impose une **contrainte référentielle** : toute valeur doit exister dans la table parent
- relie les tables entre elles : c'est elle qu'on utilise dans les **jointures**



Intégrité référentielle à l'insertion

SQL

```sql
INSERT INTO commande (NCOM, NCLI, DATECOM)
VALUES ('30200', 'Z512', '2009-01-03');
```

Résultat

✖ `Erreur : FOREIGN KEY constraint failed`

Le client `Z512` **n'existe pas** dans la table `client` : la commande ferait référence à rien.

client.NCLI existants

```
B062  B112  B332  B512  C003
C123  C400  D063  F010  F011
F400  K111  K729  L422  S127
S712
```

Z512 ? absent ✖

Règle

Insérer d'abord dans la table **parent**, ensuite dans la table **enfant**.

Intégrité référentielle à la suppression

SQL

```sql
DELETE FROM commande
WHERE NCOM = '30188';
```

Résultat

✖ `Erreur : FOREIGN KEY constraint failed`

4 lignes de `detail` référencent encore la commande 30188 (contrainte `detail.NCOM → commande.NCOM`).

Supprimer d'abord les enfants

```sql
DELETE FROM detail   WHERE NCOM = '30188';
```

✓ `OK — 4 ligne(s) affectée(s)`

```sql
DELETE FROM commande WHERE NCOM = '30188';
```

✓ `OK — 1 ligne(s) affectée(s)`

Règle (mode « blocage »)

Supprimer d'abord dans la table **enfant**, ensuite dans la table **parent**.

Que faire des lignes enfants ? ON DELETE

SQL

```sql
CREATE TABLE detail (
  NCOM TEXT NOT NULL
       REFERENCES commande (NCOM)
       ON DELETE CASCADE,
  ...
);
```

CASCADE : pratique mais dangereux

Une seule suppression peut effacer des milliers de lignes dans d'autres tables.

Option	Effet de la suppression du parent
NO ACTION	refusée (défaut)
RESTRICT	refusée immédiatement
CASCADE	enfants supprimés aussi
SET NULL	clé étrangère mise à NULL
SET DEFAULT	valeur par défaut

Même logique pour **ON UPDATE** (changement de la valeur de la clé parent).

Attention avec SQLite : activer les clés étrangères

Sans PRAGMA (comportement par défaut)

```
INSERT INTO commande VALUES ('30200', 'Z512', '2009-
01-03');
```

✓ `OK — 1 ligne(s) affectée(s)`

La commande orpheline est **acceptée** silencieusement.

Avec PRAGMA

```
PRAGMA foreign_keys = ON;
INSERT INTO commande VALUES ('30200', 'Z512', '2009-
01-03');
```

✗ `Erreur : FOREIGN KEY constraint failed`

À retenir

Dans SQLite, les clés étrangères ne sont **vérifiées** que si on exécute `PRAGMA foreign_keys = ON` **à chaque connexion**. PostgreSQL, MySQL (InnoDB)… les vérifient toujours.

Les autres contraintes

SQL

```sql
CREATE TABLE produit (
  NPRO     TEXT PRIMARY KEY,
  LIBELLE  TEXT NOT NULL UNIQUE,
  PRIX     REAL NOT NULL CHECK (PRIX > 0),
  QSTOCK   INTEGER NOT NULL DEFAULT 0
           CHECK (QSTOCK >= 0)
);
```

Contrainte	Garantit que…
PRIMARY KEY	la ligne est identifiable
FOREIGN KEY	le lien pointe vers une ligne existante
NOT NULL	la valeur est renseignée
UNIQUE	pas de doublon dans la colonne
CHECK	une condition est respectée
DEFAULT	valeur si rien n'est fourni

Lien avec le TP INCA / Ciqual

`CHECK (quantite_g > 0)` et une clé étrangère sur `code_aliment` auraient refusé dès l'insertion les anomalies que vous détectez à la main avec pandas.

03

Le langage SQL

SELECT · jointures · sous-requêtes · INSERT · UPDATE · DELETE · agrégation · NULL · transactions

SQL : Structured Query Language

DDL

Data Definition Language

Définir les structures : tables, colonnes, contraintes, index.

`CREATE · ALTER · DROP`

DML

Data Manipulation Language

Lire et modifier les données.

`SELECT · INSERT · UPDATE · DELETE`

TCL

Transaction Control

Regrouper des modifications en une unité sûre.

`BEGIN · COMMIT · ROLLBACK`

DCL

Data Control Language

Gérer les droits d'accès des utilisateurs.

`GRANT · REVOKE`

SQL est **déclaratif** : on décrit **le résultat voulu**, le SGBD choisit comment le calculer.

Créer les tables : CREATE TABLE

SQL

```sql
CREATE TABLE client (
  NCLI      TEXT PRIMARY KEY,
  NOM       TEXT NOT NULL,
  ADRESSE   TEXT NOT NULL,
  LOCALITE  TEXT NOT NULL,
  CAT       TEXT,
  COMPTE    REAL NOT NULL
);
```

- clé simple : `PRIMARY KEY` sur la colonne
- clé composite : clause `PRIMARY KEY (…)` séparée
- `CAT` sans `NOT NULL` : valeur facultative

SQL

```sql
CREATE TABLE detail (
  NCOM TEXT    NOT NULL,
  NPRO TEXT    NOT NULL,
  QCOM INTEGER NOT NULL CHECK (QCOM > 0),
  PRIMARY KEY (NCOM, NPRO),
  FOREIGN KEY (NCOM)
    REFERENCES commande (NCOM),
  FOREIGN KEY (NPRO)
    REFERENCES produit (NPRO)
);
```

Ordre de création : `client`, `produit`, `commande`, puis `detail`.

Requête de sélection : SELECT … FROM … WHERE

Syntaxe

```
SELECT liste_colonnes
FROM   nom_table
WHERE  condition;
```

SQL

```
SELECT NCLI, NOM, COMPTE
FROM   client
WHERE  COMPTE > 0;
```

- **SELECT** : quelles **colonnes** ?
- **FROM** : quelle(s) **table(s)** ?
- **WHERE** : quelles **lignes** ?

Résultat

NCLI	NOM	COMPTE
B112	HANSENNE	1250
C400	FERARD	350
K111	VANBIST	720

Opérateurs utiles :

`=` `<>` `<` `>=`

`AND` `OR` `NOT`

`BETWEEN`

`IN (…)`

`LIKE 'CS%'`

`IS NULL`

Interroger plusieurs tables : la jointure

SQL

```sql
SELECT c.NCOM, c.DATECOM, cl.NOM
FROM   commande AS c
JOIN   client   AS cl ON c.NCLI = cl.NCLI
WHERE  cl.LOCALITE = 'Poitiers';
```

Résultat

NCOM	DATECOM	NOM
30179	2008-12-22	FERARD
30184	2008-12-23	FERARD
30186	2009-01-02	FERARD

Ancienne syntaxe (équivalente)

```sql
FROM  commande c, client cl
WHERE c.NCLI = cl.NCLI AND cl.LOCALITE = 'Poitiers'
```

Attention : ne pas oublier la condition de jointure

`FROM commande, client` sans condition associe **chaque** commande à **chaque** client : 7 × 16 = **112 lignes** (produit cartésien).

Jointure externe : LEFT JOIN

SQL

```sql
SELECT cl.NCLI, cl.NOM, c.NCOM
FROM   client AS cl
LEFT JOIN commande AS c ON c.NCLI = cl.NCLI
ORDER BY cl.NCLI;
```

Clients **sans** commande (anti-jointure)

SQL

```sql
SELECT cl.NCLI, cl.NOM
FROM   client AS cl
LEFT JOIN commande AS c ON c.NCLI = cl.NCLI
WHERE  c.NCOM IS NULL;
```

→ **11 clients** n'ont passé aucune commande (lignes surlignées).

Résultat

NCLI	NOM	NCOM
B062	GOFFIN	*NULL*
B112	HANSENNE	*NULL*
B332	MONTI	*NULL*
B512	GILLET	30188
C003	AVRON	*NULL*
C123	MERCIER	*NULL*
C400	FERARD	30179
C400	FERARD	30184
C400	FERARD	30186
D063	MERCIER	*NULL*
F010	TOUSSAINT	*NULL*
F011	PONCELET	30185
F400	JACOB	*NULL*
K111	VANBIST	30178
K729	NEUMAN	*NULL*
L422	FRANCK	*NULL*
S127	VANDERKA	30182
S712	GUILLAUME	*NULL*

Sous-requête : IN (SELECT ...)

SQL

```sql
SELECT NCOM, DATECOM
FROM   commande
WHERE  NCLI IN (SELECT NCLI
                FROM   client
                WHERE  LOCALITE = 'Poitiers');
```

Résultat

NCOM	DATECOM
30179	2008-12-22
30184	2008-12-23
30186	2009-01-02

- la sous-requête produit une **liste de valeurs** (les `NCLI` de Poitiers)
- la requête principale garde les commandes dont le `NCLI` est **dans** cette liste
- se lit de **l'intérieur vers l'extérieur**

⚠ Une chaîne SQL est comparée **exactement** : `'Namur '` (avec espace) ≠ `'Namur'`.

Sous-requête ou jointure ?

Produits commandés par des clients de Toulouse

Imbriquée

```
SELECT NPRO FROM detail
WHERE NCOM IN (
  SELECT NCOM FROM commande
  WHERE NCLI IN (
    SELECT NCLI FROM client
    WHERE LOCALITE = 'Toulouse'));
```

Jointure

```
SELECT DISTINCT d.NPRO
FROM detail d
JOIN commande c ON c.NCOM = d.NCOM
JOIN client cl  ON cl.NCLI = c.NCLI
WHERE cl.LOCALITE = 'Toulouse';
```

NPRO
CS464
PA60
PS222
PA45
PH222

Même résultat

- la jointure permet d'**afficher** des colonnes des autres tables (nom du client, date…)
- la version imbriquée suit le raisonnement pas à pas
- jointure : `DISTINCT` pour éviter les doublons

NOT IN ≠ jointure avec <>

Commandes qui ne contiennent **pas** le produit PA60

NOT IN

```
SELECT NCOM FROM commande
WHERE NCOM NOT IN (SELECT NCOM
                   FROM detail
                   WHERE NPRO = 'PA60');
```

NCOM
30178
30184
30186

✓ correct

<>

```
SELECT DISTINCT c.NCOM
FROM commande c
JOIN detail d ON d.NCOM = c.NCOM
WHERE d.NPRO <> 'PA60';
```

NCOM
30178
30179
30184
30185
30186
30188

✗ faux : « contient au moins un produit autre que PA60 »

Écritures correctes équivalentes

`WHERE NOT EXISTS (SELECT * FROM detail d WHERE d.NCOM = c.NCOM AND d.NPRO = 'PA60')`, ou une anti-jointure `LEFT JOIN … WHERE … IS NULL`.

Ajouter des lignes : INSERT INTO

Syntaxe

```
INSERT INTO nom_table (liste_colonnes)
VALUES (liste_valeurs);
```

SQL

```
INSERT INTO detail (NCOM, NPRO, QCOM)
VALUES ('30185', 'PA45', 12);
```

✓ `OK — 1 ligne(s) affectée(s)`

SQL

```
INSERT INTO client
  (NCLI, NOM, ADRESSE, LOCALITE, COMPTE)
VALUES ('F422', 'FRANCK',
  '14, av. du Reclus', 'Lille', 0);
```

✓ `OK — 1 ligne(s) affectée(s)`

Attention

- les valeurs insérées doivent respecter **toutes** les contraintes (clés, `NOT NULL`, `CHECK`)
- une colonne omise reçoit `NULL` ou sa valeur `DEFAULT` (ici `CAT`)
- nombres sans guillemets ; chaînes entre apostrophes droites `'…'`

Modifier des lignes : UPDATE … SET

SQL

```sql
UPDATE produit
SET    PRIX = ROUND(PRIX * 1.05, 2)
WHERE  LIBELLE LIKE '%SAPIN%';
```

✓ `OK — 4 ligne(s) affectée(s)`

Sans WHERE, toutes les lignes sont modifiées

Toujours tester d'abord la condition avec un `SELECT … WHERE` identique, puis lancer l'`UPDATE`.

Avant

NPRO	LIBELLE	PRIX
CS262	CHEV. SAPIN 200x6x2	75
CS264	CHEV. SAPIN 200x6x4	120
CS464	CHEV. SAPIN 400x6x4	220
PS222	PL. SAPIN 200x20x2	185

Après (+5 %)

NPRO	LIBELLE	PRIX
CS262	CHEV. SAPIN 200x6x2	78.75
CS264	CHEV. SAPIN 200x6x4	126
CS464	CHEV. SAPIN 400x6x4	231
PS222	PL. SAPIN 200x20x2	194.25

Supprimer des lignes : DELETE FROM

Syntaxe

```
DELETE FROM nom_table
WHERE condition;
```

SQL

```
DELETE FROM detail
WHERE NCOM = '30184';
```

✓ `OK — 2 ligne(s) affectée(s)`

SQL

```
DELETE FROM client
WHERE NOM = 'MERCIER';
```

Deux clients s'appellent MERCIER (`C123`, `D063`) : les **deux** seraient supprimés.

Bonnes pratiques

- cibler les lignes par leur **clé primaire** : `WHERE NCLI = 'C123'`
- vérifier d'abord avec `SELECT` ; respecter les contraintes d'intégrité (enfants d'abord)

Fonctions d'agrégation

SQL

```sql
SELECT COUNT(*)              AS nb_commandes,
       COUNT(DISTINCT NCLI)  AS nb_clients
FROM   commande;
```

Résultat

nb_commandes	nb_clients
7	5

SQL

```sql
SELECT SUM(QSTOCK * PRIX) AS valeur_stock
FROM   produit
WHERE  LIBELLE LIKE '%SAPIN%';
```

Résultat

valeur_stock
650875

COUNT SUM AVG MIN MAX

Une fonction d'agrégation donne **une seule valeur** pour l'ensemble des lignes sélectionnées : on sélectionne d'abord les lignes (**WHERE**), puis on agrège.

Les données groupées : GROUP BY

LOCALITES			
NCLI	NOM	LOCALITE	COMPTE
F400	JACOB	Bruxelles	0
B332	MONTI	Genève	0
K111	VANBIST	Lille	720
S127	VANDERKA	Namur	-4580
L422	FRANCK	Namur	0
C123	MERCIER	Namur	-2300
B062	GOFFIN	Namur	-3200
S712	GUILLAUME	Paris	0
F010	TOUSSAINT	Poitiers	0
B112	HANSENNE	Poitiers	1250
C400	FERARD	Poitiers	350
C003	AVRON	Toulouse	-1700
B512	GILLET	Toulouse	-8700
F011	PONCELET	Toulouse	0
K729	NEUMAN	Toulouse	0
D063	MERCIER	Toulouse	-2250

SQL

```sql
SELECT LOCALITE,
       COUNT(*)     AS nb_clients,
       AVG(COMPTE) AS moyenne_compte
FROM   client
GROUP BY LOCALITE;
```

LOCALITE	nb_clients	moyenne_compte
Bruxelles	1	0
Genève	1	0
Lille	1	720
Namur	4	-2520
Paris	1	0
Poitiers	3	533.33
Toulouse	5	-2530

split
↓
apply
↓
combine

Sélectionner des groupes : HAVING

SQL

```sql
SELECT LOCALITE, COUNT(*) AS nb, AVG(COMPTE) AS moy
FROM   client
GROUP BY LOCALITE
HAVING COUNT(*) >= 3;
```

LOCALITE	nb	moy
Namur	4	-2520
Poitiers	3	533.33
Toulouse	5	-2530

SQL

```sql
SELECT NCLI, COUNT(*) AS nb_commandes
FROM   commande
GROUP BY NCLI
HAVING COUNT(*) >= 2;
```

NCLI	nb_commandes
C400	3

	WHERE	HAVING
Filtre	les lignes	les groupes
Moment	avant GROUP BY	après GROUP BY

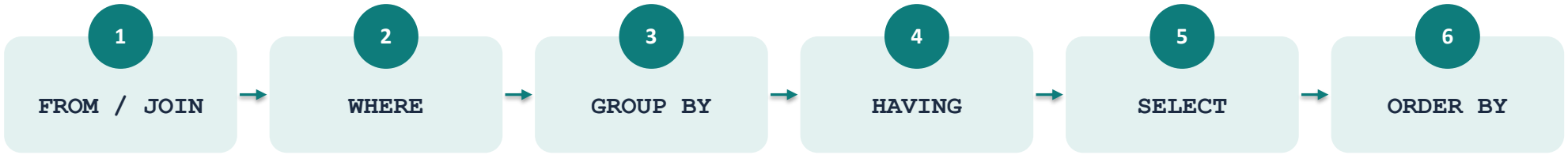
Comment lire (et écrire) une requête ?

SQL

```sql
SELECT   c.NCLI, COUNT(*) AS nb, SUM(QCOM) AS qte   -- 5
FROM     commande c                                 -- 1
JOIN     detail d ON c.NCOM = d.NCOM                -- 1
WHERE    d.NPRO LIKE 'PA%'                          -- 2
GROUP BY c.NCLI                                     -- 3
HAVING   COUNT(*) >= 2                              -- 4
ORDER BY c.NCLI;                                    -- 6
```

Résultat

NCLI	nb	qte
B512	2	92
C400	3	43



Ordre d'**évaluation** logique (≠ ordre d'écriture) : les numéros en commentaire correspondent aux étapes.

La valeur NULL : « inconnu », pas zéro

```sql
SELECT COUNT(*) FROM client WHERE CAT = NULL;
```

0

✗ toujours 0

```sql
SELECT COUNT(*) FROM client WHERE CAT IS NULL;
```

2

✓

```sql
SELECT COUNT(*), COUNT(CAT) FROM client;
```

16 / 14

COUNT(col) ignore NULL

À retenir

- tester avec `IS NULL` / `IS NOT NULL`, jamais `= NULL`
- `SUM`, `AVG`, `COUNT(col)` **ignorent** les NULL — comme pandas ignore les `NaN`

Transactions : tout ou rien

```sql
BEGIN TRANSACTION;
UPDATE client SET COMPTE = COMPTE - 500
  WHERE NCLI = 'B112';
UPDATE client SET COMPTE = COMPTE + 500
  WHERE NCLI = 'C400';
COMMIT;       -- ou ROLLBACK; en cas d'erreur
```

Les deux mises à jour sont appliquées **ensemble** ou **pas du tout** : jamais d'argent « perdu » entre les deux.

A — **Atomicité**

tout ou rien

C — **Cohérence**

les contraintes restent vraies

I — **Isolation**

les transactions concurrentes ne se gênent pas

D — **Durabilité**

une fois validé, c'est conservé (même après une panne)

04

SQL depuis Python

sqlite3 · pandas.read_sql · SQL ↔ pandas · index

Interroger une base SQLite depuis Python

Python

```python
import sqlite3
import pandas as pd

with sqlite3.connect("client_commande.db") as con:
    con.execute("PRAGMA foreign_keys = ON")
    df = pd.read_sql_query(
        """SELECT LOCALITE, COUNT(*) AS nb
           FROM client
           WHERE COMPTE < ?
           GROUP BY LOCALITE""",
        con, params=(0,))

print(df)
```

DataFrame obtenu

LOCALITE	nb
Namur	3
Toulouse	3

Paramètres « ? »

Ne **jamais** construire une requête par concaténation de chaînes (f-string) avec des valeurs saisies : risque d'**injection SQL**.

Correspondances SQL ↔ pandas

Opération	SQL	pandas
Choisir des colonnes	`SELECT a, b`	`df[["a", "b"]]`
Filtrer des lignes	`WHERE x > 0`	`df[df["x"] > 0]`
Joindre deux tables	`JOIN t2 ON t1.k = t2.k`	`df1.merge(df2, on="k")`
Jointure externe	`LEFT JOIN`	`merge(..., how="left")`
Regrouper / agréger	`GROUP BY k + SUM(x)`	`df.groupby("k")["x"].sum()`
Filtrer des groupes	`HAVING COUNT(*) >= 2`	`filtre après groupby`
Trier	`ORDER BY x DESC`	`df.sort_values("x", ascending=False)`
Valeurs distinctes	`SELECT DISTINCT k`	`df["k"].unique()`
Valeur manquante	`IS NULL`	`df["x"].isna()`
Unicité d'une clé	`PRIMARY KEY`	`df["k"].is_unique`

Différence essentielle : le SGBD **garantit** les contraintes ; pandas ne vérifie rien sans que vous le demandiez.

Accès rapide : les index

SQL

```sql
CREATE INDEX idx_commande_ncli
    ON commande (NCLI);
```

- sans index : le SGBD **parcourt toute la table** (lecture séquentielle)
- avec index : il trouve les lignes directement (arbre B, comme un dictionnaire trié)
- utile sur les colonnes de **jointure** et de **filtre** fréquentes
- coût : de la place et des écritures plus lentes

Sans index

1 000 000 lignes lues

Avec index

3 à 4 pages lues

Ordre de grandeur pour retrouver une ligne parmi 10^6 (arbre B).

05

SGBD et enjeux actuels

fondements · rôle d'un SGBD · familles de SGBD · défis

Fondements théoriques

1970	**Modèle relationnel**	E. F. Codd : tables, algèbre relationnelle (sélection, projection, jointure…)
1974	**SQL**	langage déclaratif (IBM), normalisé ISO depuis 1987
1970–80	**Normalisation**	1NF, 2NF, 3NF, BCNF : éviter redondances et anomalies
—	**Optimisation**	le SGBD réécrit la requête pour l'exécuter efficacement
—	**Monde clos (CWA)**	ce qui n'est pas dans la base est considéré comme faux

Le rôle d'un SGBD

1 Stocker

des volumes importants, de façon durable

2 Retrouver vite

index, optimisation des requêtes

3 Garantir la cohérence

contraintes, transactions

4 Partager

accès concurrents de nombreux utilisateurs

5 Protéger

droits d'accès, confidentialité

6 Récupérer

sauvegardes, reprise après panne

Familles de SGBD aujourd'hui

Relationnels

transactionnels (OLTP)

PostgreSQL · MySQL / MariaDB · Oracle · SQL Server

Embarqués

un fichier, sans serveur

SQLite · DuckDB (analytique)

Analytiques

entrepôts, colonnes (OLAP)

Snowflake · BigQuery · ClickHouse

NoSQL

documents, clé-valeur, graphes

MongoDB · Redis · Neo4j · Cassandra

SQL reste la **lingua franca** : la plupart des systèmes analytiques et même certains NoSQL proposent une interface SQL.

Les défis des bases de données

Types variés

textes, images, données spatiales, séries temporelles, JSON

Information incomplète

valeurs manquantes ou erronées : NULL, qualité des données

Volumes

milliards de lignes : performance, disponibilité, coût

Accès non-informaticiens

interfaces graphiques, langage naturel → SQL (IA)

Évolution du schéma

ajouter une colonne sans casser les programmes existants

Données distribuées

cloud, réplication, mobiles : cohérence et disponibilité

Décisionnel

entrepôts de données : de gros volumes pour une réponse courte

Sécurité et RGPD

données personnelles, droits d'accès, traçabilité

À retenir

1	**Une table, une clé**	chaque table a une clé primaire ; les clés étrangères relient les tables
2	**Le SGBD protège**	unicité, intégrité référentielle, NOT NULL, CHECK — activez les FK dans SQLite
3	**SQL est déclaratif**	FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
4	**Jointures avec soin**	toujours une condition de jointure ; LEFT JOIN pour garder les orphelins
5	**NULL ≠ 0**	IS NULL ; les agrégats ignorent les NULL
6	**SQL ≈ pandas**	JOIN ↔ merge, GROUP BY ↔ groupby : même logique, garanties différentes